Real-Time Scenario Q&A

PLAYWRIGHT · 2026 EDITION

Sreenidhi Rajakrishnan

Real-Time Scenario Interview Questions: The Playwright TypeScript Edition

How to answer the on-the-spot situational questions, 2026

Why scenario questions decide senior offers

The factual round tells the interviewer whether you know Playwright. The scenario round tells them whether they can hand you a broken pipeline on a Friday evening and trust you to make a calm, correct call. That second thing is what a senior offer is actually paying for.

A scenario question sounds like "a test passes on your laptop but fails in CI, walk me through what you do", or "the release is tomorrow and twenty tests are red, what now". There is rarely one right answer. What is being scored is your thinking under pressure: do you jump to a blind retry, or do you reach for evidence first. Do you protect the release, or do you freeze. Do you own the outcome, or do you blame the app.

Most candidates prepare facts and get surprised by scenarios. You can flip that. Every scenario below follows the same four beats, and once you have the beats, you can answer a situation you have never seen before.

The four beats to run in your head, every time.

1. Restate the situation in one line, so the interviewer knows you understood it.
2. Say what you would look at first, which is almost always evidence, the trace, the report, the CI logs, not a guess.
3. Give the structured fix with the concrete Playwright specifics, fixtures, storage state, web first assertions, sharding, network stubbing.
4. Close on the process change, so it does not happen again. Seniors fix the class of problem, not the single instance.

Read each scenario as situation, what they are really testing, how to answer, and one model line to say out loud. Say the lines out loud once before your interview. Spoken answers land differently from read ones.

Section 1. Flakiness and stability

This is the most common scenario theme, because flakiness is the thing that quietly kills trust in a suite. The interviewer wants to hear that you treat a flaky test as a defect to be diagnosed, not noise to be retried away.

A test passes on your laptop but is flaky in CI

The situation. The same spec is green every time locally and fails intermittently on the pipeline. You cannot reproduce it at your desk.

What they are really testing. Whether you debug from evidence or from guesses, and whether you understand why CI is a harsher environment than a developer machine.

How to answer. Start with the difference between the two environments, do not start changing code. CI is slower, more loaded, and runs more workers in parallel, so timing and resource contention that your fast laptop hides will surface there. My first move is evidence, not a fix. I set `trace: 'on-first-retry'` and

`screenshot: 'only-on-failure'` in the config, so the failing run leaves a full DOM, network and console recording. I open that trace and look at the exact step that failed. Nine times out of ten it is a locator resolving before the element is ready, a hard `waitForTimeout` that is long enough on my machine and short on a loaded runner, or a test depending on data another parallel test changed. The fix is almost always a web first assertion that retries, such as `await expect(locator).toBeVisible()`, in place of a fixed wait. To reproduce locally before I even push, I run with more workers and CPU throttling so my machine behaves like the runner.

What to say out loud. "I never debug a CI only failure by rerunning it. I turn on trace on first retry, open the recording of the exact failing step, and I almost always find a fixed wait or a shared data assumption. Then I reproduce it locally by cranking the worker count up to match CI."

The locators keep changing and tests break every sprint

The situation. The front end team refactors the UI often, class names and DOM structure change, and your tests break on selectors that no longer exist.

What they are really testing. Whether your locator strategy is resilient by design or brittle by habit.

How to answer. Brittle locators are a design choice, not bad luck. I lead with user facing locators, `getByRole`, `getByLabel`, `getByText`, because they track what the user sees rather than how the markup is built, and a CSS refactor rarely changes the accessible name of a button. Where the team needs a stable hook that survives redesigns, I agree `data-testid` attributes with the developers and use `getByTestId`, which turns a locator into a contract between the two teams rather than a guess. I never chain long CSS or XPath paths, because those encode the whole DOM tree and snap the moment anything above the element moves. Every locator lives inside one page object, so when something genuinely does change, it is a one line edit in one file, not a hunt across fifty specs.

What to say out loud. "I locate by role, label and text first, because those follow the user, not the markup. For anything that must be rock solid I agree a data-testid with the developers, so the locator becomes a shared contract. And all of it sits in the page object, so a change is one line in one place."

The login step itself is flaky and fails random tests

The situation. Tests fail at the very first step, the sign in, seemingly at random, and it is polluting your pass rate.

What they are really testing. Whether you still log in through the UI in every test, which is the classic mid level mistake.

How to answer. If every test logs in through the UI, you have multiplied one slow, network dependent flow across the whole suite, so any login wobble becomes suite wide flakiness. My fix is to authenticate once in a setup project, save the `storageState` to a file, and point the config at it so every test starts already signed in. No test touches the login screen unless the test is specifically about login, and those few override the state with an empty one. This removes a whole class of flakiness and takes the repeated four second login out of every run. If tokens expire mid run on a long suite, I scope the auth setup per worker and refresh the state, rather than logging in per test. The login page still gets its own dedicated tests, positive and negative, because the flow itself matters. It just stops being a tax on every other test.

What to say out loud. "I authenticate once in a setup project, save the storage state, and hand every test a signed in context through the config. The only tests that see the login screen are the ones actually testing login. That single change usually removes most of what people call flaky auth."

How do you handle retries without hiding real bugs

The situation. The interviewer asks how you use retries, probing whether you use them as a cure or a crutch.

What they are really testing. Whether you keep flakiness visible or quietly sweep it under a retry.

How to answer. A retry is a safety net, not a fix. I keep two retries in CI so a single blip does not fail a whole pipeline, but I make the report mark a test that only passed on retry as flaky, not green, so the flakiness stays on the board where someone has to look at it. A test that has been flaky for two weeks is a task, not a colour. For a test that is genuinely unstable and I cannot fix immediately, I quarantine it, tag it and move it out of the blocking suite into a non blocking lane, so it stops eroding trust in the main run while it is being investigated. What I never do is add a blanket retry and walk away, because that turns a real intermittent bug into an invisible one. The rule I say in review is that a retry buys you time to diagnose, it does not close the ticket.

What to say out loud. "I keep retries in CI, but I make the report flag a retry pass as flaky rather than green, so it stays visible. If a test is truly unstable I quarantine it into a non blocking lane and raise a ticket, rather than letting a blanket retry hide a real bug."

Long overnight runs slow down and start failing near the end

The situation. A big nightly suite runs fine for the first hour, then gets progressively slower and starts timing out towards the end.

What they are really testing. Whether you can reason about resource leaks, not just element waits.

How to answer. This pattern, fine early and failing late, points at a resource leak rather than a test bug. Browser contexts or pages that are opened and never closed pile up and starve the runner of memory. In Playwright the framework closes the per test context for you, so a leak usually means someone is manually creating extra contexts or pages, or holding a database or API connection open across tests. I confirm it by watching memory over the run, then I audit for any `browser.newContext` or `newPage` that is not paired with a close, and I move shared expensive resources into a worker scoped fixture that tears down cleanly after the worker finishes. I also cap the run by sharding it across machines so no single process carries the whole load for hours. The teardown half of a fixture, the code after the `use` call, is exactly where this gets fixed.

What to say out loud. "Fine early and failing late almost always means a leak, not a wait. I watch memory across the run, hunt for a context or page that is opened and never closed, and move anything expensive into a worker scoped fixture with real teardown after the use call."

Section 2. Speed and scale

Here the interviewer wants to know that you can keep a growing suite fast, because a slow gate is a gate people learn to skip. Answers should reach for sharding, parallelism and pushing setup down to the API.

The regression suite now takes over an hour and blocks releases

The situation. The suite has grown to a few thousand tests, a full run takes well over an hour, and it has become a bottleneck in the sprint.

What they are really testing. Whether you optimise with data or just throw more machines at it.

How to answer. I measure before I cut. I look at the per test timings in the report and find where the hour actually goes, because it is usually a small number of very slow tests, not an even spread. Then, in order: I make sure tests run fully parallel with `fullyParallel: true` and a sensible worker count, because a suite stuck at one worker is leaving most of the saving on the table. I push all setup off the UI and onto the API, since creating a record through the UI can take ninety seconds and through an API call takes two, and the UI test then only tests the one thing it is about. I reuse storage state so nothing logs in through the UI. Finally I shard across machines in CI. I also split the gates: a tagged smoke set on every pull request in a few minutes, full regression on merge. A single slow gate gets bypassed, and then you have no gate at all.

What to say out loud. "First I read the timings, because the hour is usually a handful of slow tests. Then I go fully parallel, push setup onto the API instead of the UI, reuse storage state, and shard across machines. And I split it into a fast smoke gate on every PR and full regression on merge, so the gate stays fast enough that nobody skips it."

How would you shard the suite across machines

The situation. The interviewer asks specifically how you would run one suite across several machines and still get one result.

What they are really testing. Whether you actually know the mechanics of sharding, or just the word.

How to answer. Sharding splits the test set into slices that run on separate machines at the same time. In CI I use a matrix, say three shards, and each job runs `npx playwright test --shard=1/3`, `2/3` and `3/3`, so each machine runs roughly a third of the tests. Each shard produces a blob report, and I merge those into one HTML report at the end with `merge-reports`, so the team sees a single unified result rather than three fragments. The one hard rule that makes sharding safe is test independence: no test may depend on another test running first or on shared mutable data, because two shards run with no knowledge of each other. Sharding is how you keep total wall clock time flat even as the suite keeps growing, you add a shard instead of waiting longer.

What to say out loud. "I split the run with the shard flag across a CI matrix, each machine runs its slice, and I merge the blob reports into one HTML report at the end. The precondition is that every test is independent, because the shards never see each other. When the suite grows I add a shard rather than accept a longer run."

Tests pass alone but collide when run in parallel

The situation. Each test is green run on its own, but running them in parallel produces random failures where tests seem to interfere with each other.

What they are really testing. Whether you understand test isolation and shared state.

How to answer. Collisions under parallelism nearly always mean tests are sharing mutable state, the same user account, the same order, the same row in the database, so two workers fight over it. The fix is that every test owns its own data. Each test creates what it needs through the API in a fixture, using a unique identifier such as a timestamped or randomised email, so two runs can never touch the same record. I never rely on a fixed shared account for anything a test will modify. Playwright already gives each test a fresh browser context, so client side state is isolated for free, the leak is almost always on the server side data. To catch this early I run parallel locally with a high worker count rather than discovering it only in CI. The rule I enforce in review is that no test may assume data exists before the run or that it is the only one touching that data.

What to say out loud. "Parallel collisions mean shared data, not a Playwright bug. I make every test create its own data through the API with a unique key, so no two workers touch the same record. Contexts are already isolated per test, so the fix is always on the data side."

Make the suite work on mobile and responsive layouts

The situation. The product ships a mobile web experience and the team wants the same flows covered on small screens.

What they are really testing. Whether you know Playwright can emulate devices without a whole separate framework.

How to answer. I do not stand up a second framework for mobile web. Playwright emulates devices through the config, so I add projects that spread the `devices['Pixel 7']` or `devices['iPhone 14']` descriptors, which set the viewport, user agent and touch behaviour, and the same specs run against them. Where the mobile layout genuinely differs, a hamburger menu that replaces a top nav, I keep those differences inside the page object, often with a small component that resolves the right locator per viewport, so the test stays readable. I am honest about scope in the interview: this covers mobile web and responsive layouts, and for native app testing on real devices I would reach for Appium, because Playwright is a browser tool. I also do not run every test on every device, I pick the revenue critical flows for the mobile projects and keep the full matrix on desktop, to keep the run fast.

What to say out loud. "For mobile web I add device projects in the config using the built in descriptors, so the same specs run at a phone viewport with touch. Layout differences live in the page object. For native apps I would be clear that is Appium territory, Playwright is a browser tool."

Section 3. Data and environments

These scenarios probe whether your tests can stand on their own, in any environment, without depending on data someone set up by hand or on a third party that might be down.

How do you set up and tear down test data

The situation. The interviewer asks how a test gets the data it needs and what happens to it afterwards.

What they are really testing. Whether your tests are self sufficient or quietly depend on a hand made record that will one day vanish.

How to answer. Three tiers. Static reference data that never changes, like country lists or product codes, sits in a JSON file. Data the test needs for itself is created by the test, through the API in a fixture, with a unique identifier so it can never clash with a parallel run. Genuinely shared reference data that cannot be made on demand is ring fenced, named clearly, and checked by a health check at the start of the run that fails fast if someone changed it. Teardown lives in the fixture, in the code after the `use` call, so setup and cleanup sit together and cannot drift apart. Where the environment resets cleanly between runs I lean on that rather than deleting record by record, because chasing cleanup of every row is its own source of flakiness. The rule I enforce is that no test may depend on data existing before the run. That single rule is what makes parallel execution safe and what stops the Monday morning failures after a database refresh.

What to say out loud. "Static data in a file, test specific data created through the API with a unique key, and any truly shared data ring fenced with a health check. Teardown lives right after the use call in the fixture, so setup and cleanup never drift apart. No test is allowed to assume data existed before it ran."

A third-party service your tests depend on is down

The situation. A payment gateway or an external API your flows call is unavailable, and half your suite goes red through no fault of your own.

What they are really testing. Whether you let external systems dictate your pass rate, or you control the boundary.

How to answer. My own tests should not go red because someone else's sandbox is down. For everything except the handful of tests whose whole point is the real integration, I stub the third party at the network boundary with `page.route`, returning a controlled response, so the test exercises my application's handling of that response deterministically and fast. That also lets me test the cases a live sandbox will not give me on demand, a declined card, a timeout, a malformed reply. I keep a small, clearly separated set of genuine end to end contract tests that do hit the real service, and I run those on a schedule rather than on every pull request, so a third party outage flags a real integration problem without blocking every developer's merge. The distinction I draw is between testing my code, which should never depend on their uptime, and testing the integration itself, which is a small deliberate set.

What to say out loud. "My application tests stub the third party with page.route, so a declined card or a timeout is deterministic and their downtime never turns my suite red. I keep a small set of real contract tests against the live service on a schedule, separate from the PR gate, so I still catch genuine integration breaks."

Combine an API check and a UI check in one scenario

The situation. You are asked to verify that an action taken in the UI is correctly reflected in the backend, or the reverse.

What they are really testing. Whether you can mix API and UI in one test cleanly, rather than doing everything through the browser.

How to answer. I use each layer for what it is good at. Playwright gives me an `APIRequestContext` two ways, and the difference is worth getting right because interviewers probe it. `page.request` runs inside the same browser context as the page, so it carries that context's cookies and storage state and my calls look like they came from the logged in user. The standalone `request` fixture is a separate context: it does not pick up the page's cookies, and it only has auth if I hand it one, usually through `storageState` in the config. Browser contexts are isolated from each other by design. They share the browser process and nothing else, no cookies and no storage, which is exactly what keeps parallel tests from leaking into one another. With that clear, I seed state fast, act in the UI, then assert the result at the API level, or the other way round. A typical shape is: seed an order through `page.request.post`, then in the UI confirm it appears and can be actioned, then assert through `page.request.get` that the backend state changed as expected. This is faster and more precise than clicking through five screens to reach the state, and the assertion at the data layer is less brittle than reading it back off the page. For anything the DOM does not expose, an async backend job finishing, I poll the API with `expect.poll` rather than watching the UI spin. The principle is that the UI test tests the UI, and everything around it, setup and deep verification, goes through the API.

What to say out loud. "I seed and verify through page.request, which runs in the same browser context as the page so it carries the same cookies, and I act through the UI, all in one test. The standalone request fixture is a separate context, so if I use that one I give it a storage state. Contexts share the browser process, not cookies. I assert backend truth at the API layer, which is faster and less brittle than reading it off the screen, and I poll with expect.poll for anything async the DOM does not show."

The release is tomorrow and twenty tests are red

The situation. It is the evening before a release. Twenty tests are failing. You have limited time and a decision to make.

What they are really testing. Whether you can triage under pressure and separate real risk from noise, rather than freezing or blindly rerunning.

How to answer. I do not rerun blindly and I do not panic. I triage. First I open the report and the traces and bucket the twenty failures, because twenty red tests are very often two or three real causes. A shared locator changed, a test data setup broke, an environment is pointed at the wrong place. Grouping by root cause usually collapses twenty failures into a handful of decisions. Then I sort by risk: a failure on the payment or checkout path blocks the release and gets fixed or escalated tonight, a failure on a cosmetic screen does not. For anything that turns out to be a genuinely flaky test rather than a real defect, I quarantine it out of the blocking set with a ticket, so it stops holding the release hostage while staying visible. Then I make a clear call to the release owner: here is what is a real product risk, here is what is test noise, here is my recommendation. The senior move is turning twenty scary red lines into a short, honest risk statement someone can decide on.

What to say out loud. "I bucket the twenty by root cause from the traces, because they are usually two or three real issues, not twenty. I fix or escalate anything on a revenue path tonight, quarantine genuine flakes with a ticket, and give the release owner a clear line on what is real risk versus test noise."

Section 4. Ownership and process

The final theme is the one that most separates a senior from a mid level engineer. These questions are about people, judgement and owning outcomes, not about Playwright syntax. Answer them with calm ownership.

You inherit a red pipeline you did not build

The situation. You join a team, or pick up a project, where the pipeline is already failing and the framework is unfamiliar.

What they are really testing. Whether you can get your bearings in someone else's code without blaming whoever wrote it.

How to answer. I start by reading, not rewriting. I get the suite running locally with the documented command, and if there is no such command that is my first fix, because a suite that only runs on one person's machine cannot be owned by a team. Then I read the CI logs and the artifacts, the report and traces, to separate real failures from flaky ones and from environment problems, and I bucket them the same way I would triage a release. I resist the urge to rewrite the framework, because a rewrite with no new coverage is very hard to justify and usually stalls halfway. Instead I get it green and trusted first, then improve it incrementally as I touch each area. I also ask the people who built it what they know, because their context saves me days, and it sets a collaborative tone rather than a critical one.

What to say out loud. "First I get it running locally and read the traces, I do not rewrite. I bucket the failures into real, flaky and environment, get it green and trusted, then improve incrementally as I touch each area. And I ask whoever built it what they know, because that context saves days."

How do you decide what to automate and what to leave manual

The situation. The interviewer wants your criteria for spending automation effort, not a claim that you automate everything.

What they are really testing. Whether you think about return on effort, or chase a coverage number.

How to answer. Four filters, in order. Business risk first, what loses money or trust if it breaks, so checkout and payment come before a settings screen. Then repetition, because a check that runs every release compounds its saving, and a one time migration does not. Then stability, because automating a screen that gets redesigned every sprint means rewriting it three times. Then cost to automate, because some things are cheap to verify by eye and expensive to automate, a PDF layout, a nuanced visual, a one off. What I leave manual is exploratory testing, usability, and genuinely one time checks, because a human finds things a script was never told to look for. What I say in planning is that we do not chase a coverage percentage. Ninety percent coverage of low risk screens is worse than sixty percent that covers every revenue path. I would rather report which business flows are protected than which percentage of test cases are automated.

What to say out loud. "Risk, repetition, stability, then cost to automate, in that order. I automate the flows that lose money if they break and run every release, and I leave exploratory, usability and one off checks to a human. I report which business flows are protected, not a coverage percentage."

You are asked to migrate a Selenium suite to Playwright

The situation. The team has an existing Selenium suite and wants to move to Playwright. How do you approach it.

What they are really testing. Whether you migrate with judgement or propose a risky big bang rewrite.

How to answer. I do not do a big bang rewrite, because a rewrite that adds no new coverage while it runs is very hard to justify and tends to stall halfway with two half suites and no trust in either. New tests are written in Playwright from day one, so the Selenium suite stops growing immediately. Then I port the old suite feature by feature as we touch each area, prioritising the high risk and most flaky Selenium tests first, because those are where Playwright's auto waiting and web first assertions pay off most. I do not translate line for line, I take the chance to drop the explicit waits and the brittle XPath and rebuild each flow the Playwright way, page objects and fixtures. Throughout, both suites run in CI so coverage never dips during the transition. I am also fair about Selenium in the interview, it is the right default where a large Java, TestNG and Grid investment already exists, and the migration case is resilience, speed and a better debugging story, not that the old tool was bad.

What to say out loud. "New work goes straight into Playwright so Selenium stops growing, then I port feature by feature, flakiest and highest risk first. I rebuild each flow the Playwright way rather than translating line for line, and both suites run in CI so coverage never dips. No big bang rewrite."

The team does not trust the automation

The situation. Developers and managers ignore the automated results because the suite has cried wolf too often. How do you win the trust back.

What they are really testing. Whether you understand that a suite people ignore has zero value, however clever it is.

How to answer. Trust is rebuilt with reliability and visibility, not with more tests. First I kill the noise, because a suite that is red half the time for non reasons trains everyone to ignore it. I drive the flaky rate down, quarantine what I cannot fix yet, and make sure a red build means a real problem. Then I make the

results speak the language the audience cares about. To developers, a failure comes with a trace they can open themselves, so it is evidence, not an accusation, and they can debug without my laptop. To managers, I stop reporting test counts and start reporting which business flows are protected and how quickly we catch a break, because that is the thing they actually buy. I make the suite fast and put it on every pull request, so it becomes a helpful gate rather than a slow ceremony. Trust follows a stretch of green builds that turn red only when something is genuinely broken. There is no shortcut, you earn it with a boringly reliable suite.

What to say out loud. "A suite people ignore is worth nothing, so I fix the noise first, get the flaky rate down and make red mean real. Then I give developers a trace they can open themselves and give managers protected business flows instead of test counts. Trust comes from a run of green builds that only go red for real reasons."

A reviewer disagrees with your locator strategy in code review

The situation. In a pull request, a colleague pushes back on how you have written your locators, or you are pushing back on theirs.

What they are really testing. Whether you can hold a technical position with reasons and still collaborate, rather than dig in or cave.

How to answer. I argue from principle, not preference. My position is that locators should be user facing first, `getByRole`, `getByLabel`, `getByText`, because they follow what the user experiences and survive a CSS refactor, and that a long CSS or XPath chain is brittle because it encodes the whole DOM path. If a reviewer prefers a raw CSS selector, I ask what happens when the markup around it changes, and I offer the middle ground the team can standardise on, an agreed `data-testid` for elements that need a guaranteed stable hook. The thing I care about more than winning the specific line is that the team has one written convention, so this is not re argued in every pull request. If the reviewer has a case I had not considered, I take it, because the goal is the most maintainable locator, not being right. I would rather turn a review disagreement into a one line addition to our standards doc, so the next person never has this argument.

What to say out loud. "I argue it from maintainability, role and label locators survive a refactor, long CSS chains do not, and a data-testid is the agreed hook where we need one. But the real win is getting it written into our team convention, so we settle it once instead of re arguing it in every review."

The one line to leave them with

When a scenario round winds down, close it the way you would close a real incident, calm and structured. Something like:

> "So across all of these my approach is the same: restate the problem, reach for evidence before I touch anything, fix the root cause with the right Playwright tool rather than a retry, and then change the process so it does not come back. The scenario round is not testing whether I have seen this exact situation. It is testing whether I stay calm and think in steps when I have not."

That sentence does the job. It shows the interviewer you have a repeatable method, not a bag of memorised fixes, and a repeatable method under pressure is the whole reason a senior offer exists.